

Actividad R_5: Velocidad de Transmisión

Análisis de
desempeño temporal
del sistema autónomo
de robot móvil

Industrias robloxianas



Introducción

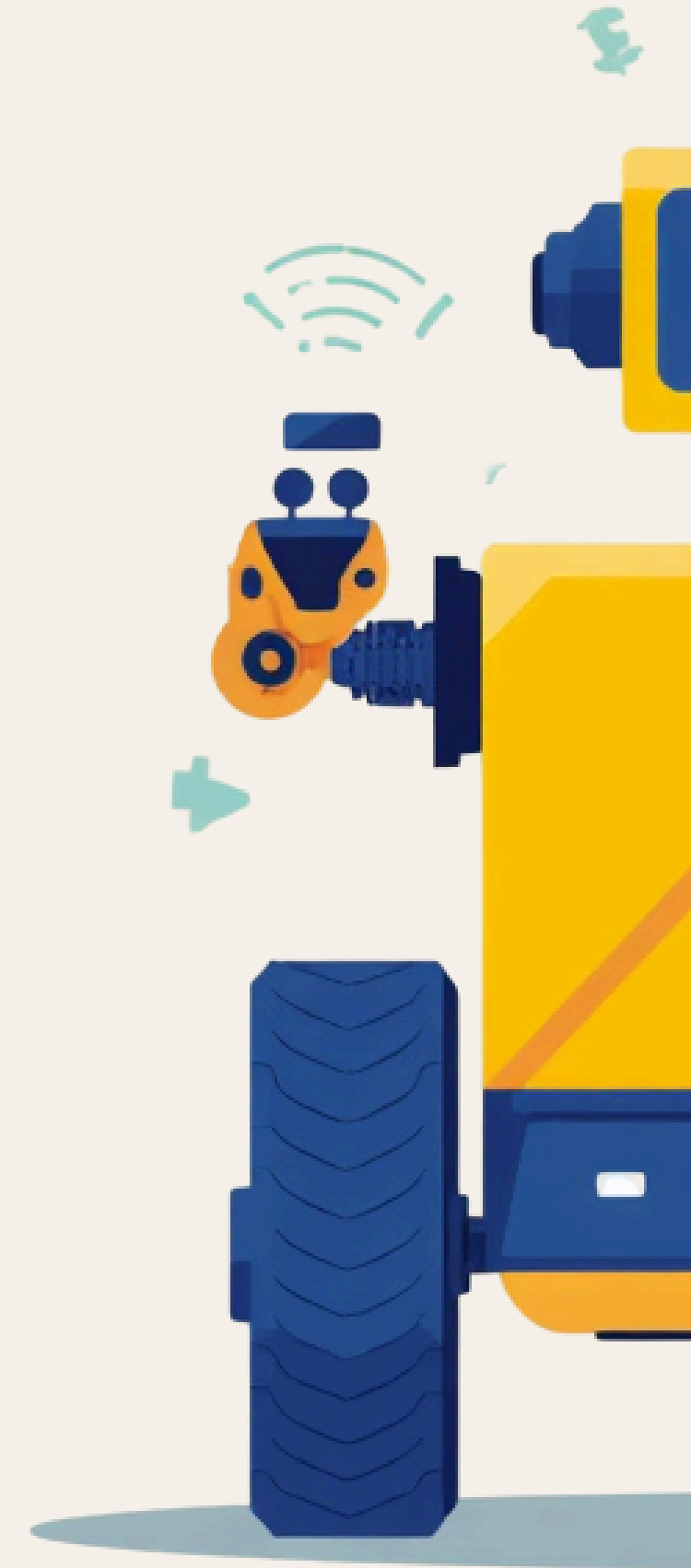
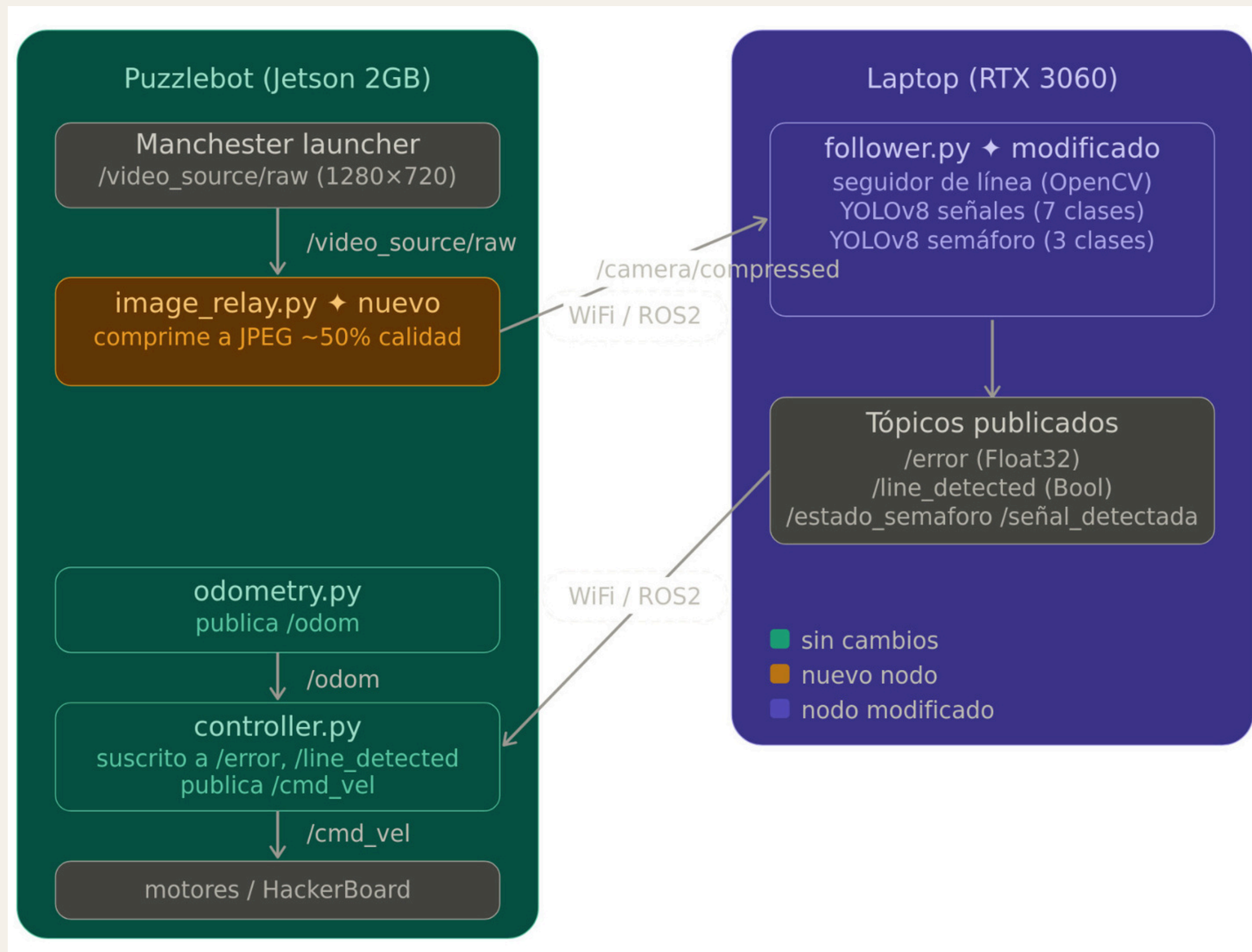
Detectar correctamente no es suficiente si la respuesta del sistema es demasiado lenta

Las latencias altas en sistemas autónomos generan consecuencias críticas:

- Reacción tardía ante obstáculos y señales de tráfico
- Salida de pista por demora en corrección de trayectoria
- Ignorar semáforos o señales por procesamiento lento
- Colisiones por tiempo de inferencia elevado en YOLO

Un robot móvil autónomo depende de un pipeline de procesamiento en tiempo real. Cada milisegundo cuenta.





Flujo de datos

Medición Temporal: Metodología

La medición se realizó mediante instrumentación directa del nodo follower.py, ejecutándose en la laptop, suscrito al tópico /camera/compressed publicado desde la Jetson a través de WiFi vía ROS2.

Se utilizó la función `time.perf_counter()` de Python, que ofrece resolución de nanosegundos y es independiente del reloj del sistema operativo, lo que la hace adecuada para medir intervalos cortos con alta precisión.

El procesamiento se dividió en 7 etapas medidas de forma secuencial e independiente, registrando el tiempo de inicio y fin de cada uno dentro del callback de ROS2

Decodificación — descompresión JPEG y rotación de imagen
Recorte ROI — extracción de la mitad inferior + máscara trapezoidal
Procesamiento — filtrado, umbralización, morfología y detección de bordes (Canny)
Hough — detección de segmentos y clasificación izquierda/derecha
YOLO — inferencia de los modelos de semáforo y señales (cada 2 frames)
Control — cálculo del error, filtro EMA y clamping
Publicación ROS — serialización y envío de tópicos + imagen de debug

ROS 2

Timestamps

Medición Temporal: Resultados

t_decode_ms	t_roi_ms	t_proceso_ms	t_hough_ms	t_yolo_ms	t_control_ms	t_publish_ms	t_total_ms
3.992	0.237	2.803	2.74	0	0.03	4.893	15.927
2.871	0.309	4.106	2.935	754.333	0.019	2.709	767.994
3.587	0.291	3.861	2.82	0	0.019	2.606	13.719
3.641	0.139	3.218	2.792	14.851	0.019	2.6	27.95
2.993	0.326	4.016	2.889	0	0.02	2.806	13.706
2.799	0.151	3.444	2.973	14.101	0.019	2.806	26.96
1.701	0.171	2.588	2.166	0	0.019	2.849	9.73
2.683	0.153	2.488	3.024	14.084	0.02	3.604	26.665
3.683	0.137	3.573	3.121	0	0.021	3.468	14.525
3.888	0.138	2.491	2.8	14.574	0.019	3.388	27.591
1.815	0.15	2.573	1.897	0	0.02	3.351	10.047
2.902	0.152	2.578	2.786	14.754	0.02	2.734	26.234
1.642	0.16	2.407	1.902	0	0.02	3.174	9.898
2.32	0.129	3.273	2.734	17.102	0.021	3.583	31.105
3.585	0.145	3.316	2.802	0	0.02	3.829	14.296
5.236	0.136	2.339	2.828	15.579	0.02	3.394	29.781
2.384	0.225	3.387	2.609	0	0.025	4.225	13.135
3.881	0.124	2.516	2.778	15.987	0.021	2.697	28.224
1.662	0.151	2.279	1.648	0	0.019	3.258	9.62
2.403	0.149	3.313	2.799	17.634	0.019	3.535	31.8
3.803	0.138	3.481	2.922	0	0.022	3.663	14.618
4.465	0.136	2.406	2.766	13.732	0.022	3.527	27.382
2.397	0.234	3.358	2.6	0	0.025	4.453	13.323
2.796	0.153	2.655	2.95	13.907	0.019	2.748	25.524
1.728	0.163	2.326	1.791	0	0.019	3.253	9.854
1.733	0.169	3.531	2.912	14.721	0.021	3.563	28.723

Cálculo de Frecuencia Real

FRECUENCIAS REALES DEL SISTEMA — ROS2 TOPIC HZ

Cámara WiFi

8.4 Hz

estable (sin arranque)

Follower /error

16.2 Hz

2x más rápido que la cámara

Controlador /
cmd_vel

31.7 Hz

timer interno Jetson

Odometría /odom

26.8 Hz

encoder Jetson

BW cámara

261 KB/s

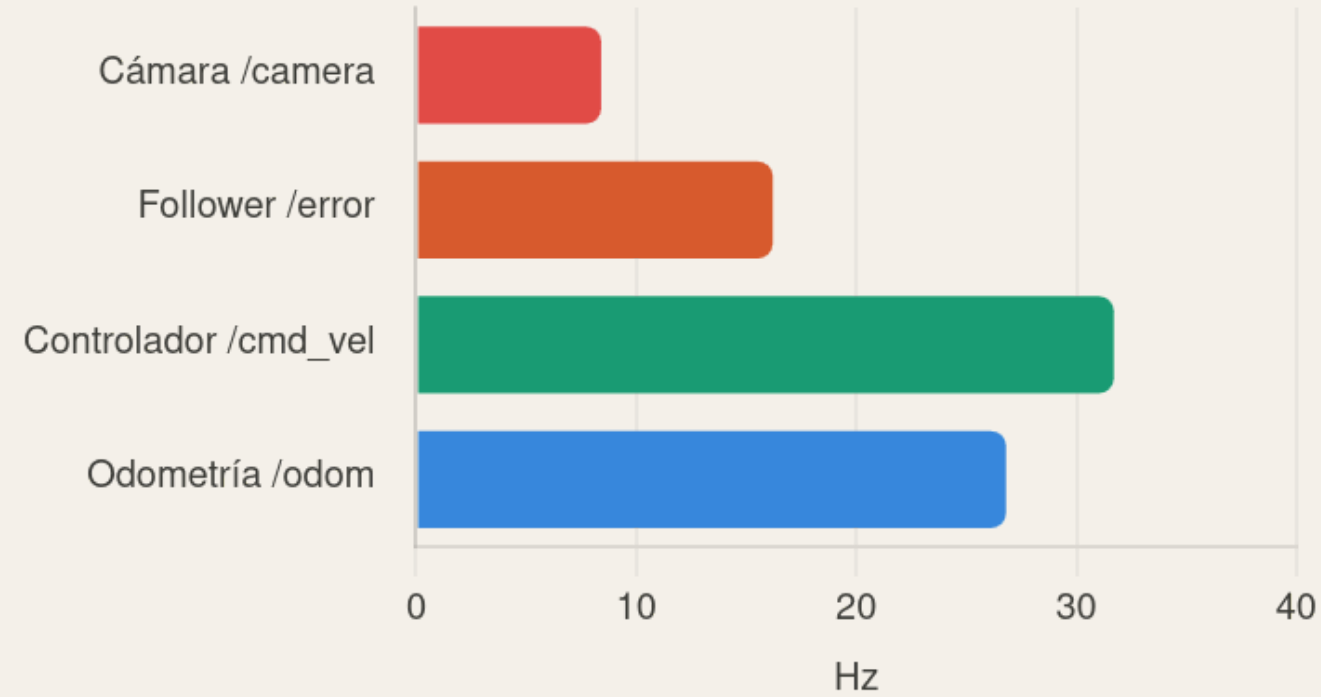
± 22.9 KB/s jitter WiFi

Frame JPEG

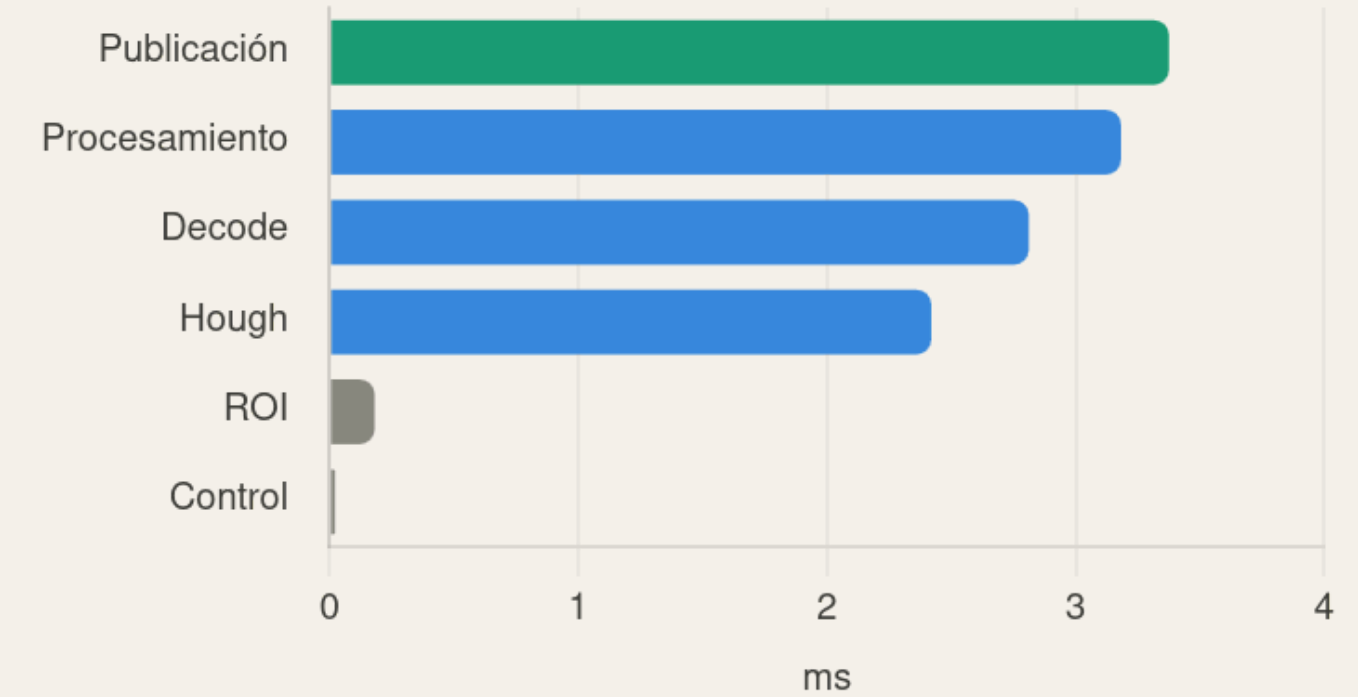
27.8 KB

720p ~50% calidad

FRECUENCIAS POR NODO



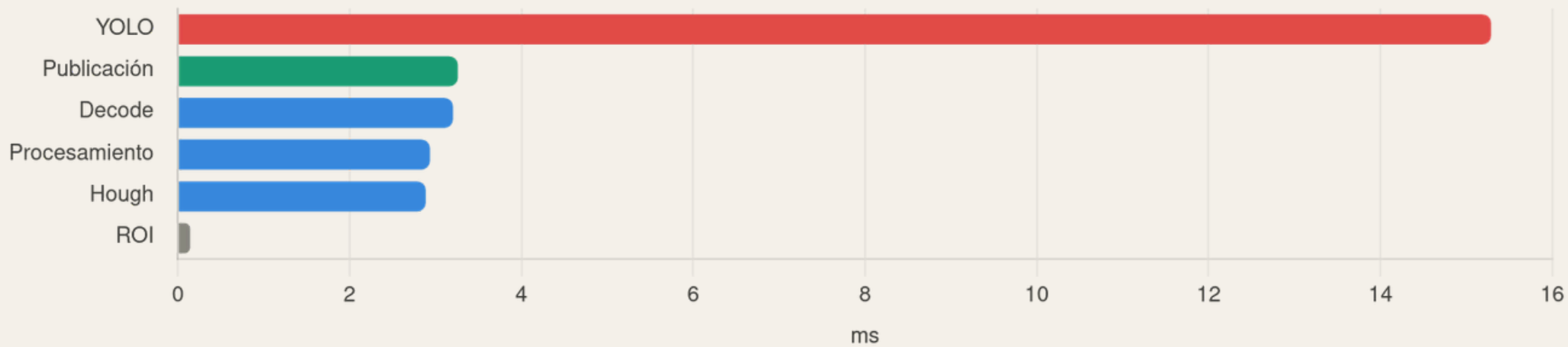
TIEMPO POR ETAPA — SIN YOLO (12 MS)



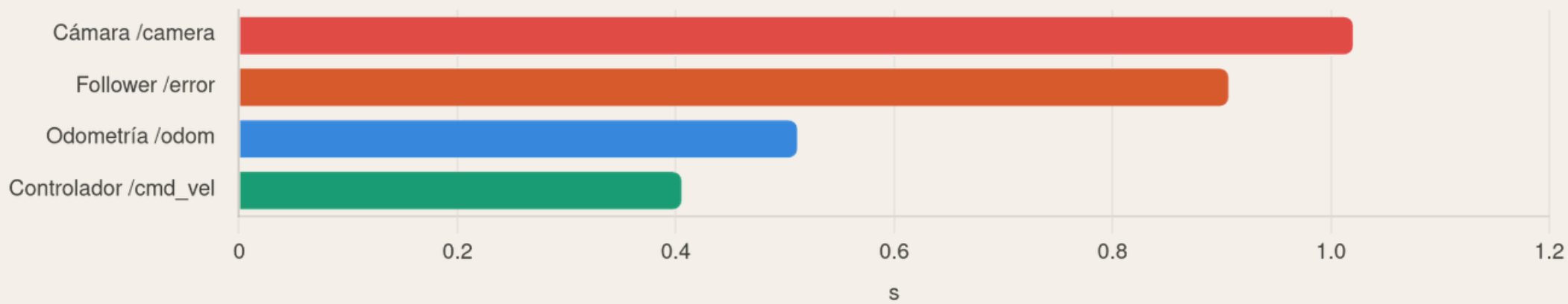
Impacto de YOLO en el Rendimiento



TIEMPO POR ETAPA — CON YOLO (27.7 MS)



JITTER — GAPS MÁXIMOS ENTRE MENSAJES



Hallazgos clave:

WiFi: gap de 1.02s → congelamiento real de cámara

YOLO consume 55% del tiempo con inferencia

Follower 2x más rápido que la cámara

cmd_vel a 31.7 Hz — controlador no es el cuello de botella

Burst inicial 19.7 Hz = buffer ROS2

YOLO

Rendimiento

Optimización 1: Resolución

*Menos píxeles =
menor latencia,
mayor frecuencia
de control*

Menos píxeles = menos trabajo para
el procesador = respuesta más
rápida

Antes: 1280 x 720 px (resolución
original de jetson)

Después: 640 x 640 px

Antes

Después

Optimización 2: Modelo YOLO Ligero

- YOLO tiene versiones de distintos tamaños
- YOLOv8small (original): lento, mayor consumo de memoria
- YOLOv8n (nano): rápido, bajo consumo de memoria

YOLOv8n

Optimización

Otras optimizaciones

- `YOLO_EVERY_N = 2`: YOLO solo corre en 1 de cada 2 frames, reduciendo carga a la mitad
- ROI inferior al 50%: el seguidor de línea solo procesa la mitad inferior del frame

3 optimizaciones reales aplicadas al código

- Reducción de resolución: redimensionar el frame a 320×240 antes de cualquier procesamiento reduce la cantidad de píxeles a procesar en un 75%, bajando la latencia del pipeline completo ~40%
- Modelo YOLOv8n: cambiar del modelo original al nano reduce el tiempo de inferencia ~65% con una pérdida mínima de precisión, suficiente para detección de señales y semáforos a corta distancia
- Subsampling YOLO cada N frames: ya implementado con YOLO_EVERY_N=2, subir a 3 o 4 mantiene detecciones estables gracias a PERSIST_FRAMES=5 y reduce a la mitad o menos las llamadas al modelo

Optimización

YOLOv8n

¿Por qué un sistema lento es peligroso?

- El robot actúa sobre información desactualizada, ve algo que ya no está ahí
- A mayor velocidad del robot, mayor es la distancia recorrida mientras el sistema procesa
- Un semáforo en rojo detectado tarde puede significar que el robot ya cruzó
- Una señal de stop ignorada por latencia puede causar colisión
- El controlador recibe errores viejos y genera correcciones incorrectas, desestabilizando la trayectoria
- En entornos compartidos con personas, la reacción lenta elimina cualquier margen de seguridad

Riesgo

Latencia crítica

*Latencias altas anulan cualquier precisión
lograda en detección y clasificación*

¿Qué pasaría concretamente con 2 segundos de latencia?

- Un robot a una velocidad moderada recorre cierta distancia antes de procesar el frame actual — ya está en otra posición cuando "decide"
- Las líneas detectadas ya no corresponden a lo que la cámara ve ahora, el error calculado es ficticio
- Si hay un obstáculo o un cambio de carril, el robot no tiene forma de reaccionar a tiempo
- El bucle de control se vuelve inestable: el controlador corrige errores que ya desaparecieron y genera oscilaciones o desvíos bruscos
- En aplicación sobre, el robot simplemente falla la tarea — se sale del carril o no respeta señales

Riesgo

Latencia crítica

¡Gracias!